UNIVERSIDAD INTERAMERICANA DE PANAMÁ

Facultad de Ingeniería de Sistemas Computacionales

PARCIAL #2

Base de Datos 2

Estudiante: Sofia Navarro (8-1017-484)
Gil Ortiz (8-1011-2311)

Fecha de Entrega: 13 de enero de 2026

Universidad Interamericana de Panamá

1. Diseño de la base de datos

Se diseñaron 6 tablas normalizadas para cubrir todos los modulos del sistema: Login, Expedientes, Aseguradoras, Tipos de Caso y Abogados.

Tabla	Descripcion	Campos principales
usuarios	Usuarios del sistema	id, nombre, username, password
aseguradoras	Companias aseguradoras	id, nombre
juzgados	Juzgados disponibles	id, nombre
tipos_caso	Clasificacion del caso	id, nombre (Transito/Penal)
abogados	Abogados de la firma	id, nombre
expedientes	Tabla principal del sistema	id, numero_caso, conductor, FKs, estado, fechas

Script SQL — Creación de tablas

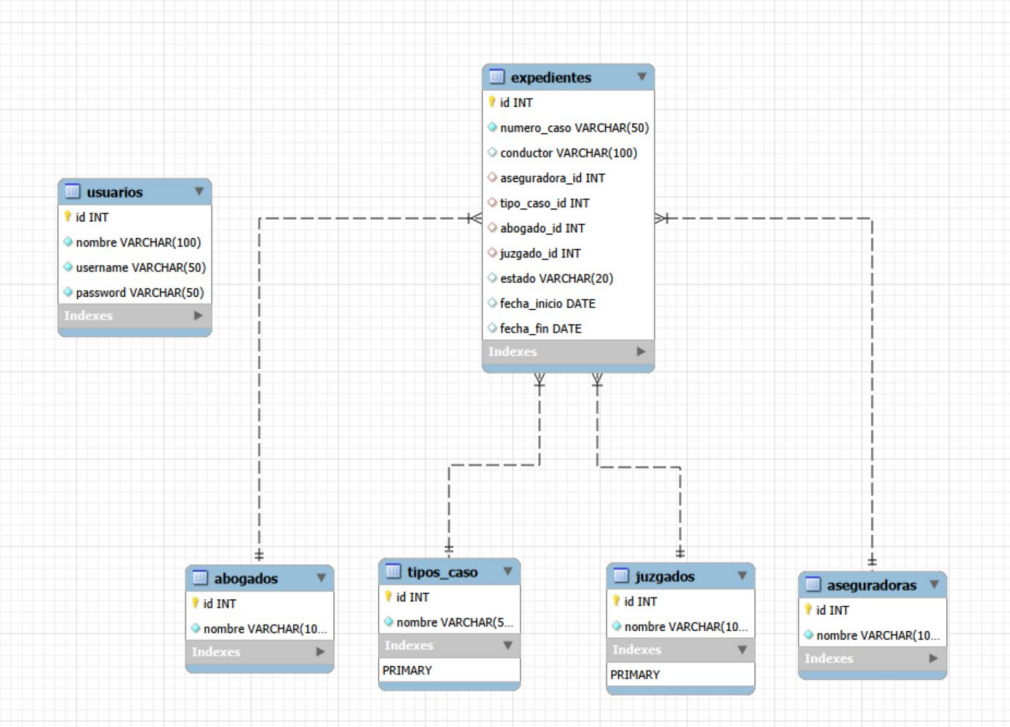
```sql
CREATE DATABASE IF NOT EXISTS delgado_abogados;
USE delgado_abogados;

CREATE TABLE usuarios (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    username VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(50) NOT NULL
);

CREATE TABLE aseguradoras (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL
);

CREATE TABLE juzgados (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL
);

CREATE TABLE tipos_caso (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL
);

CREATE TABLE abogados (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL
);

CREATE TABLE expedientes (
    id INT AUTO_INCREMENT PRIMARY KEY,
    numero_caso VARCHAR(50) NOT NULL,
    conductor VARCHAR(100),
    aseguradora_id INT,
    tipo_caso_id INT,
    abogado_id INT,
    juzgado_id INT,
    estado VARCHAR(20) DEFAULT 'Pendiente',
    fecha_inicio DATE,
    fecha_fin DATE,
    FOREIGN KEY (aseguradora_id) REFERENCES aseguradoras(id),
    FOREIGN KEY (tipo_caso_id)   REFERENCES tipos_caso(id),
    FOREIGN KEY (abogado_id)     REFERENCES abogados(id),
    FOREIGN KEY (juzgado_id)     REFERENCES juzgados(id)
);
```

127.0.0.1:5000/api/expedientes

Pretty-print


```
[
  {
    "abogado": "Lic. Diane Campbell",
    "aseguradora": "ASSA",
    "conductor": "Anthony Trejo",
    "estado": "Pendiente",
    "fecha_fin": null,
    "fecha_inicio": "Mon, 07 Jan 2019 00:00:00 GMT",
    "id": 2,
    "juzgado": "Juzgado 5TO Pedregal",
    "numero_caso": "1001",
    "tipo_caso": "Transito"
  },
  {
    "abogado": "Lic. Harold Gray",
    "aseguradora": "ANCON",
    "conductor": "Angel Delgado",
    "estado": "En curso",
    "fecha_fin": null,
    "fecha_inicio": "Mon, 07 Jan 2019 00:00:00 GMT",
    "id": 3,
    "juzgado": "Juzgado 4TO Pedregal",
    "numero_caso": "1002",
    "tipo_caso": "Transito"
  },
  {
    "abogado": "Lic. Keith Lee",
    "aseguradora": "ASSA",
    "conductor": "Martin Amado Martinez",
    "estado": "Cerrado",
    "fecha_fin": "Mon, 01 Jan 2018 00:00:00 GMT",
    "fecha_inicio": "Thu, 09 Mar 2017 00:00:00 GMT",
    "id": 5,
    "juzgado": "Juzgado 3RO Pedregal",
    "numero_caso": "1004",
    "tipo_caso": "Transito"
  },
  {
    "abogado": "Lic. Harold Gray",
    "aseguradora": "ANCON",
    "conductor": "Erick Vega",
    "estado": "Cerrado",
    "fecha_fin": "Tue, 01 Jan 2019 00:00:00 GMT",
    "fecha_inicio": "Tue, 13 Feb 2018 00:00:00 GMT",
    "id": 6,
    "juzgado": "Juzgado 4TO Pedregal",
    "numero_caso": "1005",
    "tipo_caso": "Transito"
  },
  {
    "abogado": "Lic. William Harris",
    "aseguradora": "ANCON",
    "conductor": "Guillermo Ungo",
    "estado": "Pendiente",
    "fecha_fin": null,
    "fecha_inicio": "Fri, 12 Jul 2019 00:00:00 GMT",
    "id": 8,
```


2. Diagrama de Entidad-Relación



Llaves primarias y foraneas

- Cada tabla tiene un campo 'id' como llave primaria (PK) con AUTO_INCREMENT.
- La tabla expedientes contiene 4 llaves foraneas (FK) que apuntan a las tablas catalogo.
- Las relaciones son de tipo N:1 — muchos expedientes pueden pertenecer a la misma aseguradora, juzgado, tipo de caso o abogado.

3. Normalización

Primera Forma Normal (1FN)

Una tabla está en 1FN cuando todos sus atributos contienen valores atomicos (no multivaluados) y no hay grupos repetidos.

Regla 1FN	Como se aplica en el sistema
Valores atómicos	Cada campo almacena un solo valor (nombre, fecha, estado)
Sin grupos repetidos	No hay columnas como 'abogado1', 'abogado2' — se usa una tabla separada
Llave primaria definida	Cada tabla tiene su campo 'id' como PK unica

Segunda Forma Normal (2FN)

Una tabla está en 2FN cuando está en 1FN y todos los atributos no clave dependen completamente de la llave primaria (sin dependencias parciales).

Regla 2FN	Como se aplica en el sistema
Sin dependencias parciales	El nombre de la aseguradora depende solo del id de aseguradoras, no de exp
Separacion de catalogos	aseguradoras, juzgados, tipos_caso y abogados son tablas independientes
Relacion por FK	expedientes referencia los catalogos mediante llaves foraneas

Tercera Forma Normal (3FN)

Una tabla está en 3FN cuando está en 2FN y no existen dependencias transitivas (ningún atributo no clave depende de otro atributo no clave).

Regla 3FN	Como se aplica en el sistema
Sin dependencias transitivas	El estado del expediente depende directamente del id, no de la aseguradora ni
Datos del abogado separados	El nombre del abogado vive en la tabla 'abogados', no en expedientes
Catalogos normalizados	Cada catalogo (juzgados, tipos_caso) tiene su propia tabla con su propia PK

4. Uso de Python

Se utilizo la libreria pymysql para conectar Python con MySQL. La funcion get_con() crea una nueva conexion en cada peticion para evitar errores de desconexion.

Configuracion de la conexion:

```python
from flask import Flask, jsonify, request
import pymysql

app = Flask(__name__)

def get_con():
    return pymysql.connect(
        host="localhost", user="root", password="0123456",
        database="delgado_abogados", port=3306,
        cursorclass=pymysql.cursors.DictCursor
    )
```

5. Vistas

Se implementaron 3 vistas SQL que simplifican las consultas más frecuentes del sistema

Vista 1 — v_expedientes_completos

Consolida todos los datos de un expediente en una sola consulta, uniendo las 5 tablas del sistema.

```sql
CREATE VIEW v_expedientes_completos AS
SELECT
    e.id,
    e.numero_caso,
    e.conductor,
    a.nombre  AS aseguradora,
    tc.nombre AS tipo_caso,
    ab.nombre AS abogado,
    j.nombre  AS juzgado,
    e.estado,
    e.fecha_inicio,
    e.fecha_fin
FROM expedientes e
JOIN aseguradoras a  ON e.aseguradora_id = a.id
JOIN tipos_caso   tc ON e.tipo_caso_id   = tc.id
JOIN abogados     ab ON e.abogado_id     = ab.id
JOIN juzgados     j  ON e.juzgado_id     = j.id;
```

Vista 2 — v_resumen_estados

Cuenta los expedientes por estado. Alimenta los contadores del dashboard (Pendientes 102, En curso 72, Cerrados 204).

```sql
CREATE VIEW v_resumen_estados AS
SELECT
    estado,
    COUNT(*) AS total
FROM expedientes
GROUP BY estado;
```

Vista 3 — v_expedientes_por_abogado

Muestra el total de expedientes asignados a cada abogado, clasificados por estado.

```sql
CREATE VIEW v_expedientes_por_abogado AS
SELECT
    ab.nombre AS abogado,
    COUNT(e.id) AS total_expedientes,
    SUM(CASE WHEN e.estado = 'Pendiente' THEN 1 ELSE 0 END) AS pendientes,
    SUM(CASE WHEN e.estado = 'En curso'  THEN 1 ELSE 0 END) AS en_curso,
    SUM(CASE WHEN e.estado = 'Cerrado'   THEN 1 ELSE 0 END) AS cerrados
FROM abogados ab
LEFT JOIN expedientes e ON e.abogado_id = ab.id
GROUP BY ab.id, ab.nombre;
```

6. API REST con Flask

Se implemento una API REST con Flask que expone los datos del sistema a traves de endpoints HTTP. El servidor corre en http://127.0.0.1:5000

Ejemplo — Crear expediente (POST):

URL: POST http://127.0.0.1:5000/api/expedientes
Body (JSON):
{
"numero_caso": "1010",
"conductor": "Carlos Lopez",
"aseguradora_id": 1,
"tipo_caso_id": 1,
"abogado_id": 2,

"juzgado_id": 3,
"estado": "Pendiente",
"fecha_inicio": "2026-03-01"
}
Respuesta: 201 Created — {"mensaje": "Expediente creado", "id": 11}

Ejemplo — Consultar expedientes (GET):

URL: GET http://127.0.0.1:5000/api/expedientes

Respuesta: 200 OK — Lista JSON con todos los expedientes incluyendo nombres de aseguradora, tipo de caso, abogado y juzgado (usando la vista v_expedientes_completos).

7. Lecciones aprendidas

1. La normalización permite evitar duplicidad de datos y mejora la integridad de la base de datos.
2. El uso de vistas SQL simplifica las consultas complejas en la API.
3. Flask facilita la creación rápida de APIs REST para conectar frontend y base de datos.
4. Las llaves foráneas garantizan relaciones correctas entre tablas.
5. El uso de DictCursor permite convertir los resultados en JSON fácilmente.